Explicit and Effectively Symmetric Schemes for Neural SDEs

Anonymous Author(s)
Affiliation
Address
email

Abstract

The abstract paragraph should be indented ½ inch (3 picas) on both the left- and right-hand margins. Use 10 point type, with a vertical spacing (leading) of 11 points. The word **Abstract** must be centered, bold, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

I have shortened the below by a few lines (mostly the blockquote) as space is needed for extra experiments. we probably should hint more at optimal memory

1 Introduction

Neural stochastic differential equations (SDEs) have emerged as a flexible tool for modeling stochastic dynamics, with training typically cast as a distribution-matching problem between generated and observed trajectories. Several approaches have been proposed in the literature, differing mainly in the choice of discriminating divergence. SDE-GANs [**?**] use the 1-Wasserstein distance, while Latent SDEs [**?**] optimize with respect to the KL divergence via variational inference, and can be viewed as variational autoencoders. Another alternative proposed by [**?**] trains neural SDEs non-adversarially using maximum mean discrepancies (MMD) with *signature kernels* [**? ? ?**], a recently introduced family of kernels on path space which received significant attention due to their efficiency in handling path-dependent problems [**? ? ? ?**].

While effective, the stochastic calculus underpinning SDEs can be technically cumbersome, especially when developing higher order solvers, deriving and analyzing backpropagation algorithms, or extending to rougher noises than Brownian motion. A natural way to bypass these limitations is to view SDEs through the lens of *rough paths*. Rough path theory [**? ? ?**] provides a deterministic calculus on path space that extends classical Itô integration beyond semimartingales, enabling one to treat SDEs as a special case of *rough differential equations (RDEs)*

$$\mathrm{d}y(t) \;=\; f_0(t, y_t)\,\mathrm{d}t \;+\; f(t, y_t)\,\mathrm{d}\mathbf{X}(t), \qquad y_{t_0} = y_0 \in \mathbb{R}^m, \tag{RDE}$$

where $\mathbf{X}$ is a *rough path* above a driving signal $X = (X^1, ..., X^d) : [0, T] \to \mathbb{R}^d$, $f = (f_1, ..., f_d)$ and $f_i : \mathbb{R}^m \to \mathbb{R}^m$ are vector fields for $i = 0, ..., d$ which determine the system dynamics. In fact, there is a large class of stochastic processes which can be naturally lifted to rough paths, including Gaussian processes such as fractional Brownian motion with Hurst parameter $H > 1/4$ and Volterra processes [**?**]. Viewing SDEs through the lens of RDEs yields a significant conceptual simplification. In the RDE framework, the driving signal $\mathbf{X}$ is treated as a rough path, abstracting away the stochastic integral and allowing one to work with deterministic calculus on path space. Rough path theory has become a powerful framework for analyzing modern machine learning models. It has provided the foundation for proving theoretical properties of neural differential equations [**? ? ? ?**], sequence-to-sequence architectures [**?**], and, more recently, deep selective state-space models (SSMs) [**? ? ?**] and score-based diffusion models [**?**]. For a survey of recent applications of rough path theory to machine learning, we refer the reader to [**?**].

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

From an algorithmic perspective, the RDE formulation also plays a central role in training neural SDEs (NSDEs), where one must backpropagate through an SDE solver. In particular, the derivation of the *adjoint method*—a key tool for differentiation through differential equation solvers—is considerably more transparent in the RDE setting, avoiding much of the technical machinery required in the stochastic case; see, for instance, [**?** , Sec. 3.3.4]. More broadly, the RDE viewpoint unifies backpropagation across ODEs, SDEs, and controlled differential equations (CDEs) [**?**], providing a clean pathwise calculus that is both mathematically rigorous and naturally aligned with autodiff frameworks. Several strategies have been proposed. A first, often called *discretise-then-optimise*, differentiates directly through the solvers internal operations. This yields accurate gradients and is computationally efficient, but requires storing all intermediate states and is therefore memory-intensive. A second, *optimise-then-discretise*, instead derives a backward-in-time adjoint equation and solves it numerically via a second call to the solver. This removes the need to store intermediate quantities, giving constant memory cost with respect to solver depth. However, it typically produces less accurate gradients and is slower, since the forward trajectory must be recomputed during the backward pass.

A third option leverages *algebraically reversible solvers*, which enable exact reconstruction of the solution trajectory from the terminal state, and hence permit accurate and memory-efficient backpropagation. In the setting of an autonomous ODE

$$\mathrm{d}y_t = f(y_t)\,\mathrm{d}t, \tag{1}$$

a one-step method $y_{n+1} = y_n + \Phi_h(y_n)$ is *reversible* or *symmetric* if a step of the method starting from y_1 with negative step size exactly recovers the initial condition y_0, that is, $\Phi_{-h} = \Phi_h^{-1}$. Although reversible schemes are appealing for backpropagation through differential equations, they are difficult to construct. Classical Runge–Kutta schemes are reversible only if they are implicit [**?** , Sec. 2.1], making them unsuitable for applications to Neural ODEs. More generally, symmetric parasitism-free[1] general linear methods cannot be explicit [**?**]. To avoid implicitness, existing reversible methods such as the asynchronous leapfrog integrator (ALF) [**?**] and Reversible Heun [**?**] instead track auxiliary states during integration. This yields explicit reversible schemes, but enlarges the effective state dimension and thereby reduces their practical memory advantage. These methods can also be unstable in practice on more challenging problems. For example, **?**] report:

> *"...[W]e found [Reversible Heun] is less stable compared with simple Euler integration without adjoint and results in numerical issues occasionally. ... methods without adjoint are more stable and lower loss compared with adjoint ones..." [**?**]*

A potential solution to the difficulties of reversible solvers comes with the class of *Explicit and Effectively Symmetric (EES)* Runge–Kutta schemes introduced in [**?**]. Instead of considering exactly reversible schemes, the authors propose schemes which are almost reversible, up to an acceptable tolerance level. Such schemes offer stable, explicit integration methods which are virtually indistinguishable from truly symmetric schemes in practice. The authors demonstrate the efficacy of EES schemes on classical ODEs, and show that EES schemes are capable of producing similar results to classical schemes such as RK3 and RK4. In [**?**], the formulation of EES schemes is limited to ODEs, without an explicit generalisation to the case of SDEs or RDEs. Our development shows that this framework extends beyond Euclidean ODEs, and can be realised in forms that are better suited to training: low-storage implementations, adaptive error control, and manifold-valued integration.

Derivative-free *Runge–Kutta (RK) methods* for RDEs were introduced by [**?**]. Similarly to classical RK schemes for ODEs, the study of these methods is conducted through the formalism of B-series [**? ? ?**]. A B-series is an infinite series representation of a method, indexed by (labelled) non-planar rooted trees. A brief overview of B-series is given in Appendix **??**. A natural consequence of this analysis is that a general Runge–Kutta method for RDEs is given in terms of tree-iterated integrals of the underlying driving process. In practice, these tree-iterated integrals cannot be simulated directly as their distributions are often intractable. Following [**?**], [**?**] replaced these tree-iterated integrals with products of increments of the driving path. This substitution simplifies the derivation of Runge–Kutta coefficients and makes it feasible to establish order conditions up to any desired order.

[1] Here, *parasitism* refers to spurious numerical modes of a general linear method, arising from non-principal eigenvalues and not from the true ODE dynamics, which may contaminate the computed solution.

In this paper, we formulate EES Runge–Kutta schemes for RDEs, develop practical low-storage and adaptive variants, and show that the same construction extends naturally to manifold-valued settings. We demonstrate their effectiveness for Neural SDE training. Our main contributions are as follows:

1. We extend EES Runge–Kutta schemes to the RDE setting, yielding explicit effectively symmetric integrators applicable to Neural SDEs.
2. We introduce new EES schemes, including a $2N$ low-memory schemes and FSAL methods with embedded error estimates to yield memory-efficient and error-adaptive solvers.
3. We show that this low-storage EES construction extends cleanly to homogeneous manifold integrators, yielding efficient methods for Riemannian diffusion models.
4. We establish their theoretical and empirical advantages for Neural SDE training, including improved stability over existing reversible solvers and lower gradient errors.
5. We release open-source implementations of the standard, low-memory, and manifold EES schemes in PyTorch, JAX, and Julia, facilitating adoption by the research community.

The remainder of the paper is organized as follows. Section **??** reviews reversible methods for neural differential equations, memory-efficient schemes, and their relation to manifold methods. Section 3 introduces EES schemes for ODEs, extends them to RDEs, and develops their low-storage, adaptive, and manifold-valued variants, together with convergence, stability, and backpropagation results. Section **??** evaluates the proposed methods on synthetic and real-world Neural SDE benchmarks. Section **??** concludes with limitations and future directions.

2 Preliminaries

The idea of this preliminaries is to better separate our contributions from those we take from the literature

2.1 Existing Reversible ODE and SDE Solvers

not quite clear which reversible solvers were just given for ODE vs which are for SDE. may confused reviewers A limited number of efficient reversible solvers have been proposed in the literature. The asynchronous leapfrog integrator (ALF) [**?**] for Neural ODEs overcomes the barrier of implicit schemes by tracking an additional state v as part of the integration. Applied to the ODE (1), the update rule of the ALF scheme can be written as

$$y_{n+2} = y_n + hf\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}v_n\right),$$
$$v_{n+2} = 2f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}v_n\right) - v_n.$$

For SDEs of the form $dy_t = g(t, y_t)dt + f(t, y_t)dW_t$, the Reversible Heun method [**?**] takes a similar approach, reading

$$y_{n+1} = y_n + \frac{1}{2}(g(t_n, v_n) + g(t_{n+1}, v_{n+1}))\Delta t + \frac{1}{2}(f(t_n, v_n) + f(t_{n+1}, v_{n+1}))\Delta W_n,$$
$$v_{n+1} = 2y_n - v_n + g(t_n, v_n)\Delta t + f(t_n, v_n)\Delta W_n.$$

The Reversible Heun method is efficient, requiring only one evaluation of the drift g and the diffusion f per step. However, the method is known to be inherently unstable [**?** , Theorem D.19].

To ameliorate this instability, **?**] propose a method transforming any ODE integration scheme $y_{n+1} = y_n + \Psi_h(t, y_n)$ into one which is reversible and with often larger stability domain than ALF and reversible Heun. They take the update

$$y_{n+1} = \lambda y_n + (1 - \lambda)v_n + \Psi_h(t, v_n),$$
$$v_{n+1} = v_n - \Psi_{-h}(t_{n+1}, y_{n+1}),$$

for a given coupling parameter $\lambda \in (0, 1]$. Nonetheless, the resulting stability domain remains much smaller than the that of the underlying method Ψ, and additionally on the coupling parameter λ.

In all cases, these methods break the strict RK structure, so they do not necessarily admit clean manifold generalizations as stnadard RK schemes do via RKMK. Furthermore, they must expand the state space, making them non memory optimal and so on

2.2 EES Schemes for ODEs

EES schemes [**?**] are a class of explicit Runge–Kutta methods which offer an efficient approach to reversible integration without compromising stability, while retaining the standard explicit Runge–Kutta form. Given positive integers $m \geq n$, an explicit Runge–Kutta scheme Φ_h is said to be an $\mathrm{EES}(n, m)$ scheme if Φ_h is of order n and $\Phi_{-h} \circ \Phi_h$ recovers the initial condition up to order m when applied to an ODE. When m is large, such schemes exhibit near-reversible behaviour, which is often sufficient in practice. In [**?**], Butcher tableaux are derived for 3-stage $\mathrm{EES}(2, 5)$ and 4-stage $\mathrm{EES}(2, 7)$ schemes. Although the emphasis there is mostly on $\mathrm{EES}(2, 7)$, for our Neural SDE applications we restrict attention to $\mathrm{EES}(2, 5)$, since we do not expect the additional accuracy of $\mathrm{EES}(2, 7)$ to justify the extra stage in our setting. Proposition 2.1 gives the general one-parameter family of $\mathrm{EES}(2, 5)$ tableaux, which we denote by $\mathrm{EES}(2, 5; x)$.

Proposition 2.1 ([**?** , Proposition 8.4]). For $x \in \mathbb{R} \setminus \{1, \pm\frac{1}{2}\}$, 3-stage Runge–Kutta schemes belonging to $\mathrm{EES}(2, 5)$ have Butcher tableau

$$\begin{array}{c|ccc}
0 & & & \\
\dfrac{1+2x}{4(1-x)} & \dfrac{1+2x}{4(1-x)} & & \\
\dfrac{3}{4(1-x)} & \dfrac{(4x-1)^2}{4(x-1)(1-4x^2)} & \dfrac{1-x}{(1-4x^2)} & \\
\hline
 & x & \dfrac{1}{2} & \dfrac{1}{2} - x
\end{array} \tag{2}$$

The stability region for $\mathrm{EES}(2, 5)$ is comparable to classical methods such as Kutta's RK4, significantly larger than those of ALF and Reversible Heun whose region is the line $[-i, i]$. Theorem 2.1 gives the exact form of this region for $\mathrm{EES}(2, 5; x)$.

[placeholder: stability_regions_1]

Theorem 2.1. Suppose that, for some $x \neq 1, \pm\frac{1}{2}$, $\mathrm{EES}(2, 5; x)$ is used to obtain a solution $\{y_n\}_{n\geq 0}$ to the linear test equation $dy = \lambda y\, dt$, where $\lambda \in \mathbb{C}$ and $y_0 \neq 0$. Then $y_n \to 0$ as $n \to \infty$ if and only if

$$\left|1 + \rho + \frac{1}{2}\rho^2 + \frac{1}{8}\rho^3\right| < 1, \qquad \rho = \lambda h.$$

Figure 1: *Stability domain for* $\mathrm{EES}(2, 5; 1/10)$ *compared to Kutta's RK3 and RK4.*

This follows by a direct computation of the stability function $R(\rho) = 1 + \rho + \frac{1}{2}\rho^2 + \frac{1}{8}\rho^3$, which is independent of x. Following **?** , Section 8.1], we fix $1/10$ and refer to $\mathrm{EES}(2, 5; 1/10)$ as *the* $\mathrm{EES}(2, 5)$ scheme. we should plot rev Heun and ALF as a line from -i,i somehow.

Beyond stability and near-reversibility, EES(2,5) has an additional structural property we exploit in the sequel, namely, that it permits realization as a $2N$ scheme.

3 Explicit and Effectively Symmetric (EES) schemes for RDEs

3.1 A $2N$ realization of $\mathrm{EES}(2, 5)$

The key structural observation used in our construction is that the chosen $\mathrm{EES}(2, 5)$ scheme is a Williamson-type $2N$ Runge–Kutta method. Recall that a $2N$ method admits a recurrence of the form

$$\begin{aligned} \Delta Y_i &= A_i \Delta Y_{i-1} + h\, f(Y_{i-1}), \\ Y_i &= Y_{i-1} + B_i \Delta Y_i, \end{aligned} \qquad i = 1, \ldots, s. \tag{3}$$

Thus, for a state of size N, an s-stage implementation requires only $2N$ registers, compared with $(s + 1)N$ for a standard explicit Runge–Kutta method.

From the tableau 2.1 it is clear that $\mathrm{EES}(2,5;x)$ admits such a low memory representation. Using the above memory complexities, this formulation **halves the memory requirement** of $\mathrm{EES}(2,5;x)$ from $4N$ to $2N$. We formalize this with the following proposition. The two-register recurrence structure also enables lifting to a homogeneous manifold integrator, which we utilize in Section 4.3.

Theorem 3.1 ([**?** , Theorem 2]). An explicit Runge–Kutta method is a Williamson $2N$ method iff

$$a_{ij}(b_{j-1} - a_{j,j-1}) = (a_{i,j-1} - a_{j,j-1})b_j, \qquad i = 3,\dots,s, \quad j = 2,\dots,i-1.$$

Proposition 3.1. For every admissible parameter $x \in \mathbb{R} \setminus \{1, \pm\frac{1}{2}\}$, the method $\mathrm{EES}(2,5;x)$ is a Williamson $2N$ scheme.

The proof in Appendix C.1 proceeds by substituting the coefficients of Proposition 2.1 into the criterion of Theorem 3.1; since $\mathrm{EES}(2,5;x)$ is 3-stage, only one identity requires verification, which holds for all admissible x. The recurrence (3) serves as the starting point for the manifold construction in Section 4.3. In the context of neural ODEs and SDEs, the $2N$ formulation is particularly advantageous: standard discretize-then-optimize schemes must retain all intermediate solver states for backpropagation, whereas reversibility permits recomputation of forward states during the backward pass, ensuring that only $2N$ registers are required at any point in time.

3.2 Simplified Runge–Kutta Methods for RDEs

We consider rough differential equations (RDEs) of the form

$$dy_t = f(y_t)\, d\mathbf{X}_t, \tag{4}$$

where $\mathbf{X}$ is an α-Hölder branched rough path, $\alpha \in (0,1]$, and f is smooth and bounded with bounded derivatives; see Appendix B.3 for background. Following **?**], we assume $\mathbf{X}$ is geometric, i.e. there exist smooth approximations $\{X^h\}_{h>0}$ whose natural lifts $\{\mathbf{X}^h\}_{h>0}$ satisfy $\varrho_\alpha^g(\mathbf{X}^h, \mathbf{X}) = \mathcal{O}(h^{r_0})$ for some $r_0 > 0$, where ϱ_α^g is the inhomogeneous geometric rough path metric. For convergence rates of such approximations for Gaussian processes, see [**?**]. Letting y^h denote the solution to (4) driven by $\mathbf{X}^h$, a simplified Runge–Kutta scheme for y^h on an equidistant grid with stepsize h is given by

$$\begin{aligned} y_{n+1}^h &= y_n^h + \sum_{m=1}^{d}\sum_{i=1}^{s} b_i f_m(k_i) X_{t_n,t_{n+1}}^{(m)}, \\ k_i &= y_n^h + \sum_{m=1}^{d}\sum_{j=1}^{s} a_{ij} f_m(k_j) X_{t_n,t_{n+1}}^{(m)}, \end{aligned} \tag{5}$$

where $X_{t_n,t_{n+1}}^{(m)}$ denotes the m-th component increment of X^h over $[t_n, t_{n+1}]$. Convergence rates for schemes of the form (5) from **?**] are recalled in Appendix **??**.

3.3 Stability of EES schemes for SDEs

As discussed in the introduction and subsequently in Section 3, EES schemes offer a stable alternative to reversible integration. Whilst the stability of EES in the case of ODEs has been studied in **?**] and Section 3, we are interested in the stability of $\mathrm{EES}_{\mathcal{R}}$ when applied to stochastic drivers for our applications to Neural SDEs. To evaluate the stability in the context of SDEs, we consider the *mean-square stability*, which has been widely used for the analysis of stochastic integration methods in the literature [**? ? ? ? ? ? ? ? ? ?**]. Given the test equations

$$dy_t = \lambda y_t dt + \mu y_t dW_t, \tag{6}$$

where $\lambda, \mu \in \mathbb{C}$ and $y_0 \neq 0$ almost surely, a solution $\{y_n\}_{n\geq 0}$ derived from a numerical integrator is said to be *mean-square stable* if $\lim_{n\to\infty} \mathbb{E}(|y_n|^2) = 0$. It follows in a similar fashion to Theorem 2.1 that $\mathrm{EES}(2,5;x)$ applied to 6 is mean-square stable if and only if

$$\mathbb{E}\left[\left|1 + \rho + \frac{1}{2}\rho^2 + \frac{1}{8}\rho^3\right|^2\right] < 1,$$

where $\rho = \lambda dt + \mu dW_t \sim N(\lambda dt, \mu^2 dt)$. Figure 2 shows 4 cross-sections of the stability domain. For comparison, we take the RDE analogues of RK3 and RK4 in (5), $\mathcal{R}$(RK3) and $\mathcal{R}$(RK4). Along most cross-sections, $\mathrm{EES}_{\mathcal{R}}(2,5)$ achieves similar or greater stability than $\mathcal{R}$(RK3) and $\mathcal{R}$(RK4).

[placeholder: stoch_stability]

Figure 2: *Cross sections of the mean-square stability domains of* $\mathrm{EES}_{\mathcal{R}}(2,5)$*,* $\mathcal{R}$*(RK3) and* $\mathcal{R}$*(RK4).*

4 Experiments

we should discuss what experiments to keep

4.1 Baselines

4.2 Synthetic

4.3 Manifold SDE

?] conjectured that $2N$-storage Runge–Kutta schemes correspond naturally to low-storage, minimal-exponential commutator-free (CF) homogeneous manifold integrators of the form introduced by **?**]. Consequently, a Euclidean $2N$ $\mathrm{EES}(2,5)$ immediately yields a homogeneous manifold integrator with minimal memory *and* compute requirements, $\mathrm{CF\text{-}EES}(2,5)$.

We evaluate on the task of one-step-ahead covariance forecasting over $\mathrm{SPD}(n)$: given a context window of W consecutive empirical covariance matrices $\Sigma_{t-W+1}, \ldots, \Sigma_t$, the model predicts Σ_{t+1}.

A GRU encoder reads the context window and produces a conditioning vector $c \in \mathbb{R}^{d_c}$. Rather than lifting to a flat tangent space, we integrate a neural SDE *intrinsically* on $\mathrm{SPD}(n)$, initialised at the most recent observation $Y_0 = \Sigma_t$. The drift and diffusion are parameterised directly in the geometric frame of $T_{Y_s}\mathrm{SPD}(n)$, so positive definiteness is preserved exactly throughout integration and the predicted covariance $\hat{\Sigma}_{t+1} = Y_T$ is guaranteed to lie on the manifold. Integration is carried out with the CFEES25 solver [**?**], a structure-preserving reversible solver that admits efficient adjoint gradient computation.

Models are trained by minimising MSE in a local chart at the projected training mean, with an optional affine-invariant Riemannian distance regulariser weighted by $\beta \geq 0$, and evaluated on the

mean geodesic distance $\overline{d}_{\mathrm{R}} = \frac{1}{N}\sum_i d_{\mathrm{R}}(\hat{\Sigma}^{(i)}, \Sigma^{(i)})$, which is coordinate-free and invariant under congruence transformations. Full experimental details are given in Appendix **??**.

4.4 Torus SDE

Each RNA nucleotide has seven backbone dihedral angles $(\alpha, \beta, \gamma, \delta, \epsilon, \zeta, \chi)$ that naturally live on the flat d-torus $\mathbb{T}^d = \mathrm{SO}(2)^d$, the simplest non-trivial compact Lie group. We train a neural SDE on $\mathbb{T}^7$ for one-step-ahead torsion-angle forecasting: given a 20-residue context window, predict the next residue's seven dihedrals. A GRU encoder over $(\sin\theta, \cos\theta)$ angle features and nucleotide identity (A/C/G/U one-hot) produces a conditioning vector; drift and diffusion MLPs parameterise the SDE whose retraction is the componentwise wrap modulo 2π (Figure 3). Training minimises the wrapped energy score [**?**], a strictly proper scoring rule on the torus. We train on 67,000 non-canonical residue pairs (those whose torsion vector lies $> 1\,\mathrm{rad}$ from the training circular mean) drawn from 3,000 BGSU representative RNA chains at 3.0 Å resolution, and evaluate with 16-sample MC averaging.

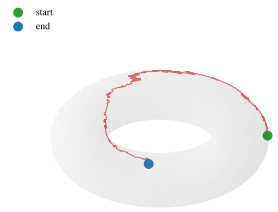

Figure 3: Stochastic trajectory on $\mathbb{T}^2$ embedded in $\mathbb{R}^3$, integrated by CF-EES(2,5) for 600 steps.

On the non-canonical test set ($n = 17{,}682$), the model achieves an overall wrapped MAE of $0.663\,\mathrm{rad}$ against $0.807\,\mathrm{rad}$ for the training circular mean — an $\mathbf{17.9}\%$ relative reduction — and beats the baseline on all seven angles (Figure 4). The largest margins are on the two most multimodal backbone torsions: α at $+34.6\%$ (gauche$^+$/trans split) and γ at $+22.2\%$ (backbone crankshaft motion).

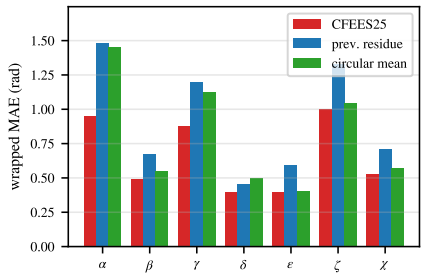

Figure 4: Per-angle wrapped MAE on non-canonical test residues ($n = 17{,}682$). CF-EES(2,5) (red) beats both the training circular mean (green) and the predict-previous baseline (blue) on every angle.

Because the torus is a flat abelian Lie group, this experiment cleanly isolates the $\mathcal{O}(1)$ memory claim from the noncommutative geometry exercised by the SPD companion. At state dimension $d = 350$ and batch 32, CF-EES$(2,5)$ under `ReversibleAdjoint` stays constant at $46.5\,\mathrm{MiB}$ of XLA scratch across $n_{\mathrm{steps}} \in [5, 800]$, while CG2 and CG4 under one-snapshot-per-step checkpointing grow linearly at $0.043\,\mathrm{MiB}$/step (Figure 5). The ratio reaches $1.72\times$ at $n_{\mathrm{steps}} = 800$ and extrapolates to $10.2\times$ at 10,000. Full architecture and dataset details are in Appendix E.1.

References

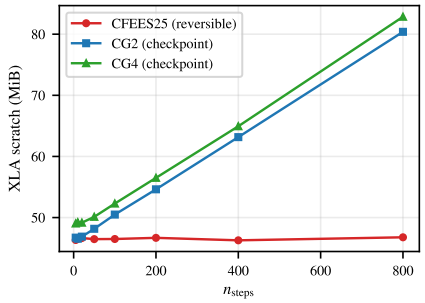

Figure 5: XLA compile-time scratch for one forward+backward step at $d = 350$, batch 32. CF-EES(2,5) under `ReversibleAdjoint` (red) is flat at $46.5\,\mathrm{MiB}$; CG2/CG4 under checkpointing grow linearly.

A Homogenous Space Integrators and Commutator-free $\mathrm{EES}(2,5;x)$

This section introduces geometric numerical integration over homogeneous spaces as an extension of the well-known Lie group case and describe the SPD geometry of our experiments. We then provide some background on commutator-free methods, of which our $\mathrm{CF}-\mathrm{EES}(2,5;x)$ belongs, and show its memory-compute optimality.

A.1 Symmetric Integration over Homogenous Spaces

For chart-based Homogeneous space integrators, selfadjointness depends not only on the Butcher tableau of the underlying method, but also requires the chosen local coordinates are compatible with time reversal. There are two natural ways to enforce this:

(i) use symmetric coordinates, centered at a geodesic or flow midpoint, in the style of self-adjoint Lie-group methods [**?**],

(ii) or use an embedded frozen-flow model whose frozen dynamics already integrate to an exactly reversible map on M.

The first route is intrinsic, but generally leads to midpoint-centered constructions and hence away from the present explicit frozen-flow framework. The second route is the one used here, which works for arbitrary homogeneous spaces.

Consider a homogeneous space M endowed with a transitive action $\Lambda\colon G\times M\to M$ of a Lie group G. Rather than working with vector fields on M directly, the key idea is to represent them through elements of the Lie algebra $\mathfrak{g}=T_eG$: each $v\in\mathfrak{g}$ acts on M via the **fundamental vector field**

$$v_M(y) \coloneqq \frac{\mathrm{d}}{\mathrm{d}t}\bigg|_{t=0}\Lambda(\exp(tv),\,y), \tag{7}$$

the infinitesimal version of the group action. A vector field F on M is then represented through a state-dependent generator $\xi(y)\in\mathfrak{g}$ via $F(y)=\xi(y)_M(y)$, replacing the role that Lie-algebra multiplication vy plays on the group itself:

$$gh \rightsquigarrow \Lambda(g,y), \qquad vy \rightsquigarrow v_M(y), \tag{8}$$

for $g\in G$, $v\in\mathfrak{h}_y/\mathfrak{g}$, $y\in M$. This substitution lifts standard Lie-group integration schemes from G to M. In general, this construction is only unique up to the isotopy algebra $\mathfrak{h}_y=\{A\in\mathfrak{g}\colon A_m(y)=0\}$, the set of generators whose induced vector field at y is zero and thus produce no tangent motion. An example is given in Example A.1. To avoid this one fixes a representative of each tangent direction, typically by choosing a complement to the isotropy algebra and lifting only through that subspace.

Example A.1 (Example of degenerate choices due to the isotropy algebra)**.** Consider the sphere $S^2\simeq\mathrm{SO}(3)/\mathrm{SO}(2)$ with the standard transitive action of $\mathrm{SO}(3)$ on $S^2\subset\mathbb{R}^3$. At the north pole $p=e_3$, the isotropy subgroup is

$$H_p=\{R\in\mathrm{SO}(3): Rp=p\}\simeq\mathrm{SO}(2),$$

namely the rotations about the vertical axis. Its Lie algebra consists of the infinitesimal rotations that leave p fixed to first order.
Equivalently, if $K\in\mathfrak{so}(3)$ generates a rotation about the e_3-axis, then its fundamental vector field vanishes at p:

$$K_{S^2}(p)=\frac{\mathrm{d}}{\mathrm{d}t}\bigg|_{t=0}\exp(tK)p=0.$$

Hence, if $A\in\mathfrak{so}(3)$ represents some tangent vector at p, then so does $A+K$ for any such K, since

$$(A+K)_{S^2}(p)=A_{S^2}(p)+K_{S^2}(p)=A_{S^2}(p).$$

Thus the lift of a tangent vector from T_pS^2 to $\mathfrak{so}(3)$ is not unique: one may add any element of the isotropy algebra without changing the induced tangent motion at p.

The reverse of a frozen homogeneous-space flow is obtained by changing the sign of t. Indeed, for a fixed generator $v \in \mathfrak{g}$, by the group action identity Λ gives

$$\Lambda(\exp(-tv), \Lambda(\exp(tv), y_0)) = \Lambda(\exp(-tv)\exp(tv), y_0) = \Lambda(e, y_0) = y_0. \tag{9}$$

Hence the frozen flow is exactly reversible: its adjoint is obtained by time reversal, that is, by replacing t with $-t$. This is a property of the frozen flow itself, before any recomputation of the generator along the trajectory.

A.2 Construction of Self-adjoint SPD Geometry

We now make our construction concrete for the SPD experiment of Section 4.3. The SPD manifold $\mathcal{S}^n_{++} \simeq \mathrm{GL}(n)/\mathrm{O}(n)$ carries the transitive $\mathrm{GL}(n)$-action

$$\Lambda(g, P) = gPg^\top, \tag{10}$$

so the natural algebra for generating tangent directions is $\mathfrak{gl}(n)$. The network outputs $x \in \mathbb{R}^{n(n+1)/2}$, which is mapped to a symmetric matrix $A \in \mathrm{Sym}(n) \subseteq \mathfrak{gl}(n)$, a Lie-algebra element that acts on the current state P to produce a tangent vector. Applying the fundamental vector field construction gives

$$A_{\mathcal{S}_{++}}(P) = \frac{\mathrm{d}}{\mathrm{d}t}\bigg|_{t=0} e^{tA} P e^{tA^\top} = AP + PA^\top, \tag{11}$$

which, since $A^\top = A$, reduces to $AP+PA$. The vector field on $\mathcal{S}^n_{++}$ is therefore $F(P) = AP+PA$, with $A \in \mathfrak{gl}(n)$ playing the role of $\xi(P)$. Freezing this field and integrating exactly yields

$$\dot{P} = AP + PA \implies P(t) = e^{tA} P_0 e^{tA}, \tag{12}$$

the Lie-group flow on $\mathcal{S}^n_{++}$ corresponding to the generator A.

The adjoint of this *frozen* flow is obtained by reversing the sign of t, giving

$$P(-t) = e^{-tA} P e^{-tA^\top}. \tag{13}$$

For fixed A, this map is the exact inverse of the forward frozen flow. Indeed,

$$e^{-tA}\left(e^{tA} P e^{tA^\top}\right)e^{-tA^\top} = \left(e^{-tA}e^{tA}\right) P \left(e^{tA^\top} e^{-tA^\top}\right) = IPI = P. \tag{14}$$

Hence the frozen congruence flow on $\mathcal{S}^n_{++}$ is self-adjoint, with a particularly simple exact inverse.

A.3 Commutator-Free Integrators

Commutator free (CF) [**?**] generalize Crouch-Grossman (CG) [**?**] methods by allowing multiple exponentials per stage, while still avoiding the explicit evaluation of commutators as in Runge–Kutta Munthe–Kaas methods [**?**].

For simplicity, we present the autonomous case; the non-autonomous case is handled by the standard augmentation in time. Given internal stage values $Y_i \in M$, we evaluate the left-trivialized stage slopes

$$K_i = f(Y_i) \in \mathfrak{g}, \qquad i = 1, \ldots, s. \tag{left-trivialized slope}$$

Each stage is then formed by acting on the current state y_n with an ordered product of exponentials of linear combinations of previously computed slopes

$$Y_i = \Lambda\left(\mathcal{T}\left\{\prod_{l=1}^{L_i} \exp\left(h \sum_{j=1}^{i-1} a_{l;ij}\, K_j\right)\right\}, y_n\right), \qquad i = 1, \ldots, s. \tag{stage equation}$$

where Y_i is the stage value and y_{n+1} is the next solution point. The step update then takes the form,

$$y_{n+1} = \Lambda\left(\mathcal{T}\left\{\prod_{l=1}^{L} \exp\left(h \sum_{i=1}^{s} \beta_{l;i}\, K_i\right)\right\}, y_n\right), \tag{update equation}$$

where L_i and L denote the numbers of exponentials used in stage i and in the final update, respectively, and $\mathcal{T}$ denotes ordered multiplication in G, with smaller indices appearing to the right.

A.4 Adjoint Sensitivities for Commutator-Free Neural ODEs

The continuous adjoint system underlying CFEES is obtained by applying **?** , Theorem 4.5] through the homogeneous-space lift introduced in Section A.1 and specialized in Section A.2. In the SPD case, this requires no further construction beyond the fixed symmetric lift $A \in \mathrm{Sym}(n)$.

A.5 Theoretical Compute and Memory Requirements of Lie Group Integrators

standardise flops per eval, and per exp. Then with fixed per-eval flops, compare total cost due to number of exps and evals (decomposed), then same for memory The dominant cost of Lie group integrators is the evaluation of exponentials. In an s-stage Crouch–Grossman (CG) method, stage i uses $i-1$ exponentials and the final update uses s more, giving

$$N_{\exp}^{\mathrm{CG}}(s) = \sum_{i=1}^{s}(i-1) + s = \frac{s(s+1)}{2}.$$

Thus, the exponential count of CG grows quadratically with the number of stages.

A general commutator-free (CF) method instead exponentiates linear combinations of previously computed stage slopes. Writing L_i for the number of exponentials used in stage i and L for the number used in the final update, its total exponential count is

$$N_{\exp}^{\mathrm{CF}}(s) = \sum_{i=1}^{s} L_i + L.$$

CG is recovered as the special case $L_i = i-1$ and $L = s$. In practice, CF methods are designed so that L_i and L remain small, reducing the exponential count from quadratic to linear in s. For example, the 3-stage and 4-stage commutator-free schemes of Celledoni, Marthinsen, and Owren require 3 and 5 exponentials per step, respectively.

A particularly efficient instance is the 2N-storage commutator-free form of Bazavov, which reuses exponentials across stages and requires exactly one exponential per stage. Its per-step exponential count is therefore

$$N_{\exp}^{\text{2N-CF}}(s) = s,$$

while the storage remains fixed at two registers.

Table 1: Per-step exponential counts for s-stage Lie group integrators. We denote the methods of [**?**] CMO CF.

Method	$s=3$	$s=4$	$s=5$
CG	6	10	15
CMO CF3/CF4	3	5	–
2N-CF	3	4	5

B Rooted Trees and B-Series for RDEs

This section requires additions: Planarly branched rough paths, a brief description of manifold rough integration. or maybe we just hope no one asks? this could be a lot for an appendix. We may wish to place this before the diffgeo section above too

B.1 The Connes-Kreimer Hopf Algebra

We give a brief account of non-planar (labelled) rooted trees and the Connes-Kreimer Hopf algebra. We refer the reader to [**?**] for a comprehensive presentation. A non-planar labelled rooted tree is defined as a graph $\tau = (V, E, r)$ with vertex set V, edge set E and a root vertex $r \in V$, together with a set of vertex decorations drawn from $\{1, \dots, d\}$. We denote the empty tree by $\emptyset$. Given trees $\tau_1, \dots, \tau_m$, we write $[\tau_1, \dots, \tau_m]_a$ to denote the tree formed by connecting the root vertices of $\tau_1, \dots, \tau_m$ to a new root, which receives the label $a \in \{1, \dots, d\}$. Non-planarity means that the

order of the trees in $[\tau_1, \ldots, \tau_m]_a$ is irrelevant. Repeated trees will be denoted using power notation, for instance

$$[\tau_1, \tau_1, \tau_2, \tau_3, \tau_3, \tau_3]_a = [\tau_1^2, \tau_2, \tau_3^3]_a.$$

We write $|\tau|$ to denote the number of vertices in a tree. Additionally, we define the following combinatorial quantities, defined on unlabelled trees:

$$\emptyset! = 1, \quad \bullet! = 1, \quad [\tau_1, \ldots, \tau_m]! = |[\tau_1, \ldots, \tau_m]| \prod_{i=1}^{m} \tau_i!,$$

$$\sigma(\emptyset) = 1, \quad \sigma(\bullet) = 1, \quad \sigma([\tau_1^{k_1}, \ldots, \tau_m^{k_m}]) = \prod_{i=1}^{m} k_i! \sigma(\tau_i)^{k_i},$$

$$\beta(\emptyset) = 1, \quad \beta(\bullet) = 1, \quad \beta([\tau_1^{k_1}, \ldots, \tau_m^{k_m}]) = \binom{[\tau_1^{k_1}, \ldots, \tau_m^{k_m}]}{|\tau_1|, \ldots, |\tau_m|} \prod_{i=1}^{m} \frac{1}{k_i!} \beta(\tau_i)^{k_i}.$$

We will refer to the commutative juxtaposition of trees as a forest. We write $\mathcal{T}$ to denote the set of all non-planar labelled rooted trees, and $\mathcal{T}_N \subset \mathcal{T}$ to denote the trees τ with $|\tau| \leq N$. The free commutative $\mathbb{R}$-algebra generated by $\mathcal{T}$ will be denoted $\mathcal{H}$. The Connes-Kreimer [**?**] Hopf algebra on $\mathcal{H}$ is defined as follows. Multiplication $\mu : \mathcal{H} \otimes \mathcal{H} \to \mathcal{H}$ is defined as the commutative juxtaposition of two forests, extended linearly to $\mathcal{H}$. The multiplicative unit is defined to be the empty forest $\emptyset$. The counit map $\varepsilon : \mathcal{H} \to \mathbb{R}$ is defined by $\varepsilon(\emptyset) = 1$ and $\varepsilon(\tau) = 0$ for all non-empty trees $\tau \in \mathcal{H}$. The coproduct map is defined recursively by

$$\Delta(\emptyset) = \emptyset \otimes \emptyset,$$
$$\Delta[\tau_1, \ldots, \tau_m]_a = [\tau_1, \ldots, \tau_m]_a \otimes \emptyset + (\mathrm{id} \otimes B_+^a)(\Delta\tau_1 \cdots \Delta\tau_m),$$

where $B_+^a(\tau_1 \cdots \tau_m) := [\tau_1 \cdots \tau_m]_a$ for a forest $\tau_1 \cdots \tau_m$. The definition is extended to a linear multiplicative map on $\mathcal{H}$. We will occasionally use Sweedler's notation

$$\Delta\tau = \sum_{(\tau)} \tau^{(1)} \otimes \tau^{(2)}$$

for the coproduct. We omit the definition of the antipode S here, and instead refer the reader to [**? ?**]. We denote the dual of the Connes-Kreimer Hopf algebra by $\mathcal{H}^*$. For $\varphi_1, \varphi_2 \in \mathcal{H}^*$, the convolution product is defined by

$$\varphi_1 * \varphi_2 = \mu_{\mathbb{R}} \circ (\varphi_1 \otimes \varphi_2) \circ \Delta,$$

with $\mu_{\mathbb{R}} : \mathbb{R} \otimes \mathbb{R} \to \mathbb{R}$ denoting multiplication in $\mathbb{R}$.

B.2 B-Series Expansions of ODEs

For any tree $\tau \in \mathcal{T}$, the so-called elementary differential $F(\tau)(y)$ [**?**] is defined recursively by

$$F(\emptyset)(y) = y, \quad F(\bullet_i)(y) = f_i(y),$$
$$F([\tau_1, \tau_2, \ldots, \tau_m]_i)(y) = f_i^{(m)}(y)(F(\tau_1)(y), F(\tau_2)(y), \ldots, F(\tau_m)(y)).$$

Given a map $\varphi : \mathcal{T} \to \mathbb{R}$, the associated B-series is defined

$$B_h(\varphi, y_0) := \sum_{\tau \in \mathcal{T}} \frac{h^{|\tau|}}{\sigma(\tau)} \varphi(\tau) F(\tau)(y_0).$$

A key property of B-series is their closure under composition. One can show that for two B-series [**? ?**],

$$B_h(\varphi_2, B_h(\varphi_1, y_0)) = B_h(\varphi_1 * \varphi_2, y_0),$$

where $\varphi_1 * \varphi_2$ is the convolution product defined above. It can be shown that the exact solution to the ODE in (1) admits a B-series representation $y(h) = B_h(e, y_0)$, where $e(\tau) = 1/\tau!$. Similarly,

the solution given by a Runge–Kutta scheme with coefficients $\{a_{ij}\}_{1\le i,j\le s}$, $\{b_i\}_{1\le i\le s}$ admits the B-series representation $B_h(\varphi, y_0)$, where [**?** , Lemma 312B]

$$\varphi(\tau) := \sum_{i_1,\dots,i_n} b_{i_1} \prod_{(k,\ell)\in E} a_{i_k,i_\ell}$$

for a tree $\tau = (V, E, r)$ with $|\tau| = n$.

We refer the reader to [**? ? ?**] for a detailed account of B-series and the Butcher group.

B.3 Branched Rough Paths

Let $\mathcal{H}$ be the Connes-Kreimer Hopf algebra of non-planar labelled rooted trees defined above.

Definition B.1. Let $\alpha \in (0, 1]$. An α-Hölder branched rough path is a map $\mathbf{X} : [0, T]^2 \to \mathcal{H}^*$ such that

1. for all $s, t \in [0, T]$ and $\tau_1, \tau_2 \in \mathcal{H}$,
$$\langle \mathbf{X}_{s,t}, \tau_1\rangle\langle \mathbf{X}_{s,t}, \tau_2\rangle = \langle \mathbf{X}_{s,t}, \tau_1\tau_2\rangle,$$
2. for all $\tau \in \mathcal{H}$,
$$\langle \mathbf{X}_{s,t}, \tau\rangle = \sum_{(\tau)} \langle \mathbf{X}_{s,u}, \tau^{(1)}\rangle\langle \mathbf{X}_{u,t}, \tau^{(2)}\rangle,$$
where $\Delta\tau = \sum_{(\tau)} \tau^{(1)} \otimes \tau^{(2)}$.
3. for all $\tau \in \mathcal{H}$,
$$\sup_{s\neq t} \frac{|\langle \mathbf{X}_{s,t}, \tau\rangle|}{|t - s|^{\alpha|\tau|}} < \infty.$$

Remark. As remarked in [**? ?**], the components $\langle \mathbf{X}_{s,t}, \tau\rangle$ with $|\tau| > N$ are determined by those with $|\tau| \le N$, where N is the largest integer such that $N\alpha \le 1$.

The space of α-Hölder branched rough paths is a complete metric space under the metric

$$\varrho_\alpha(\mathbf{X}, \mathbf{Y}) := \sum_{\tau\in\mathcal{T}_N} \sup_{s\neq t} \frac{|\langle \mathbf{X}_{s,t} - \mathbf{Y}_{s,t}, \tau\rangle|}{|t - s|^{\alpha|\tau|}},$$

where $N = \lfloor 1/\alpha \rfloor$.

B.4 Manifold rough RK

1. Redmann-Riedel rough RK is valid for any euclidean RK scheme
2. EES25 is a 2N RK scheme (normal tableau just with an extra constraint)
3. EES25 is thus also a manifold scheme
4. Redmann-Riedel still valid

C Proofs and Additional Theorems

C.1 Proof of Theorem 3.1: All $\mathrm{EES}(2, 5; x)$ schemes are $2N$.

Proof. Since $\mathrm{EES}(2, 5; x)$ is a 3-stage explicit Runge–Kutta method, it is enough to verify the relation given in Theorem 3.1.. From the tableau of 2.1,

$$a_{21} = \frac{1 + 2x}{4(1 - x)}, \qquad a_{31} = \frac{(4x - 1)^2}{4(x - 1)(1 - 4x^2)}, \qquad a_{32} = \frac{1 - x}{1 - 4x^2}, \qquad b_1 = x, \qquad b_2 = \frac{1}{2}.$$

Direct substitution gives

$$a_{32}(b_1 - a_{21}) = \frac{1 - x}{1 - 4x^2}\left(x - \frac{1 + 2x}{4(1 - x)}\right) = \frac{4x^2 - 2x + 1}{4(2x - 1)(2x + 1)},$$

and

$$(a_{31} - a_{21})b_2 = \frac{1}{2}\left(\frac{(4x-1)^2}{4(x-1)(1-4x^2)} - \frac{1+2x}{4(1-x)}\right) = \frac{4x^2-2x+1}{4(2x-1)(2x+1)}.$$

Thus 3.1 holds identically in x, and $\mathrm{EES}(2,5;x)$ is Williamson $2N$ for every admissible x. □

C.2 The Method $\mathrm{EES}(2,7;\frac{1}{4}(2-\sqrt{2}))$ is Also $2N$

While the main text focuses on $\mathrm{EES}(2,5;x)$ methods, our approach extends naturally to $\mathrm{EES}(2,7;\frac{1}{4}(2-\sqrt{2}))$, which may better preserve gradient stability under especially long rollouts.

We first specialize **?** , Theorem. 2] to the four-stage case with the following proposition.

Theorem C.1 ([**?** , Theorem 2]). For a 4-stage explicit Runge–Kutta method, the Williamson $2N$ conditions are

$$a_{32}(b_1 - a_{21}) = (a_{31} - a_{21})b_2, \tag{15}$$
$$a_{42}(b_1 - a_{21}) = (a_{41} - a_{21})b_2, \tag{16}$$
$$a_{43}(b_2 - a_{32}) = (a_{42} - a_{32})b_3. \tag{17}$$

Theorem C.2. The method $\mathrm{EES}(2,7;\frac{1}{4}(2-\sqrt{2}))$ is a distinguished $2N$ scheme.

Proof. For $\mathrm{EES}\big(2,7;\frac{1}{4}(2-\sqrt{2})\big)$ one has

$$a_{21} = \frac{1}{2}(2-\sqrt{2}), \quad a_{31} = 0, \quad a_{32} = \frac{1}{2}\sqrt{2}, \quad a_{41} = \frac{1}{2}(2-\sqrt{2}), \quad a_{42} = 0, \quad a_{43} = \frac{1}{2}\sqrt{2},$$

$$b_1 = \frac{1}{4}(2-\sqrt{2}), \qquad b_2 = \frac{1}{4}\sqrt{2}, \qquad b_3 = \frac{1}{4}\sqrt{2}.$$

Then

$$a_{32}(b_1 - a_{21}) = \frac{1}{2}\sqrt{2}\left(\frac{1}{4}(2-\sqrt{2}) - \frac{1}{2}(2-\sqrt{2})\right) = \frac{1}{4}(1-\sqrt{2}),$$

while

$$(a_{31} - a_{21})b_2 = \left(0 - \frac{1}{2}(2-\sqrt{2})\right)\frac{1}{4}\sqrt{2} = \frac{1}{4}(1-\sqrt{2}),$$

so (15) holds. Next,

$$a_{42}(b_1 - a_{21}) = 0 = (a_{41} - a_{21})b_2,$$

so (16) holds. Finally,

$$a_{43}(b_2 - a_{32}) = \frac{1}{2}\sqrt{2}\left(\frac{1}{4}\sqrt{2} - \frac{1}{2}\sqrt{2}\right) = -\frac{1}{4},$$

and

$$(a_{42} - a_{32})b_3 = \left(0 - \frac{1}{2}\sqrt{2}\right)\frac{1}{4}\sqrt{2} = -\frac{1}{4},$$

so (17) holds. Therefore $\mathrm{EES}\big(2,7;\frac{1}{4}(2-\sqrt{2})\big)$ is also Williamson $2N$. □

Remark. For a Williamson 2N scheme, the number of additional 2N constraints is $\frac{(s-1)(s-2)}{2}$. Thus, for a 2-stage explicit RK method, $\frac{(2-1)(2-2)}{2} = 0$, and the 2N scheme is trivial.

D Additional Experiments

D.1 Stochastic Volatility

We embed all stochastic volatility models in the RDE framework of **?**], which spans classical BlackScholes [**?**] to contemporary rough volatility models [**?**].

D.2 Molecular Dynamics

Differentiable MD enables end-to-end force-field optimization by backpropagating through simulated trajectories, making it possible to fit against experimental observables, such as vibrational spectra, that reflect ensemble behaviour rather than static energy surfaces. The main challenge is that this requires differentiation through long Langevin trajectories, leading to memory costs linear in the rollout length and motivating specialized adjoints or checkpointing to remain tractable.

In this experiment, we fit an embedded neural network (EANN) 64-molecule water force field to experimental liquid-phase IR spectra, showing that $\mathrm{EES}(2,5)$ enables adjoint differentiation with $\mathcal{O}(1)$ memory in the rollout length for the full Langevin state $y_t = (r_t, v_t) \in \mathbb{R}^{6N_{\mathrm{atom}}} = \mathbb{R}^{1152}$. We adapt the deterministic setup of **cite** by replacing the velocity Verlet integrator with SDE solvers, while retaining the original dataset and training objective, now posed as a mean minimization problem.

We simulate finite-temperature Langevin trajectories and optimize the EANN parameters by matching the resulting infrared spectrum to experiment via the weighted squared loss over B batches

$$\mathcal{L}_{\mathrm{IR}}^{(B)}(\theta) = \frac{1}{B}\sum_{b=1}^{B}\left[10\sum_{\tilde{\nu}<1000\,\mathrm{cm}^{-1}}\left(I_\theta^{(b)}(\tilde{\nu}) - I^{\exp}(\tilde{\nu})\right)^2 + \sum_{1000\le\tilde{\nu}\le 4000\,\mathrm{cm}^{-1}}\left(I_\theta^{(b)}(\tilde{\nu}) - I^{\exp}(\tilde{\nu})\right)^2\right].$$

Here I_θ is computed from dipole autocorrelations along the simulated trajectory, so resolving the low-frequency bands requires differentiation through long stochastic rollouts of large state vectors.

We

Thus, EES integrators make tractable training even with extremely large state dimensions over long rollouts.

Table 2: Neural SDE training results across stochastic volatility models.

Method	**Problem**	**W1 Loss**				**Runtime (s)**	**Gradient error**
		64	**128**	**192**	**256**		
Reversible Heun	Black–Scholes						
	Bergomi						
	Heston						
	Local						
	Rough Heston						
	Quadratic rough Heston						
	Rough Bergomi						

E Experimental and Implementation Details

E.1 Additional Details for Torus SDE Experiments

Model. The state $\theta \in \mathbb{T}^7$ is integrated via the SDE $\mathrm{d}\theta_\tau = f(\theta_\tau, c)\,\mathrm{d}\tau + g(\theta_\tau, c)\,\mathrm{d}W_\tau$, initialised at the last observed angle $\theta_0 = \theta_t$. The drift f is a 4-layer MLP (width 384, ReLU) and the diffusion g is a 3-layer MLP (width 384, ReLU, final layer softplus at bias -2.0, scale 0.05). Both take $(\sin\theta, \cos\theta, c) \in \mathbb{R}^{14+64}$ as input, where $c \in \mathbb{R}^{64}$ is the GRU encoder output. The encoder processes per-step features $(\sin\theta, \cos\theta, \mathrm{onehot}(b))$ through a single-layer GRU (hidden 384).

Dataset. Representative RNA chains come from the BGSU non-redundant list at 3.0 Å resolution, parsed from mmCIF via `biotite`. We retain contiguous runs of ≥ 30 residues with all seven dihedrals defined. Train/val/test splits are by chain-level deterministic hash (70/15/15%). We filter to *non-canonical* targets: residues whose 7-dimensional torsion vector lies $> 1\,\mathrm{rad}$ from the training circular mean (approximately 44% of the data). With `max_chains`$= 3{,}000$ this yields 67,262 training and 17,682 test pairs.

Training. Wrapped energy score loss with $m = 4$ samples, AdamW at peak LR 10^{-4} with cosine decay (5% end), warmup 1,000 steps, gradient clipping at norm 1, 20 epochs. Test-time predictions are circular-meaned over 16 MC rollouts.

Table 3: XLA compile-time scratch (MiB) for one forward+backward step at $d = 350$, batch 32.

n_{steps}	CF-EES(2,5)	CG2	CG4	CG2/CF-EES
5	**46.3**	46.7	49.0	1.01×
10	**46.5**	46.5	49.2	1.00×
20	**46.6**	46.9	49.2	1.01×
50	**46.5**	48.2	50.1	1.04×
100	**46.5**	50.5	52.3	1.09×
200	**46.7**	54.6	56.5	1.17×
400	**46.3**	63.2	64.9	1.37×
800	**46.8**	80.4	82.8	1.72×

Memory scaling (Table 3).

Table 4: Per-angle wrapped MAE (rad) on 17,682 non-canonical test residues.

Angle	CF-EES(2,5)	Circular mean	Prev. residue	Lift vs. mean
α	**0.951**	1.454	1.481	+34.6%
β	**0.491**	0.551	0.671	+10.9%
γ	**0.874**	1.123	1.197	+22.2%
δ	**0.400**	0.499	0.456	+19.8%
ϵ	**0.394**	0.405	0.593	+2.7%
ζ	**1.000**	1.043	1.329	+4.1%
χ	**0.528**	0.574	0.712	+8.0%
Overall	**0.663**	0.807	0.920	+17.9%

Per-angle forecasting accuracy (Table 4).

E.2 Additional Details for SPD Manifold Experiments

Daily closing prices for `AAPL`, `JPM`, `XOM`, `JNJ`, and `PG` are sourced from Yahoo Finance over the period 2005-01-01 to 2025-12-31, from which log-returns are computed. Rolling 5×5 empirical covariance matrices are then estimated using a 20-day window with Ledoit–Wolf shrinkage to improve conditioning [**?**]. This results in 5 260 matrices, which are split 70/15/15% across time.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We provide an algorithm blocks for backpropagation, the Butcher tableaux of each method, and a complete description of the geometry for manifold experiments.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All code for our reversible integrations, baseline models, and experiments is provided in the anonymous GitHub repositories.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.

- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.

- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

Justification: Our experiments do not employ crowdsourced data or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.